

BRAC University

Department of Electrical & Electronic Engineering

Course: EEE-103 (Computer Programming)

Section: 02

Semester: Summer 2026

Assignment 2

Instructions:

- Answer all questions carefully.
- Write clean, well-indented C programs.
- Use appropriate comments where necessary.
- For tracing questions, a proper tracing table must be shown.

Q1.

Write a **C program** that stores **N integer numbers** entered by the user using **dynamic memory allocation**. After storing the values, modify the data such that the first element is exchanged with the last element, the second element is exchanged with the second last element and so on.

Next, increase the size of the allocated memory by **one additional element** and store the value **103** at the last position. Finally, print the modified array. You must use dynamic memory (Except for N).

Q2.

Write a **C program** to implement a function that searches for a given number in an integer array and returns its index. Define a function named **findIndex** that takes an integer array and a number as input and returns the index of the number if it is found in the array. If the number is not found, the function should return **-1**.

In the **main()** function, declare an integer array:

```
int marks[] = {3, 5, 90, 2, 1};
```

Prompt the user to enter a number to search for, call the **findIndex** function, and print an appropriate message based on the returned value.

Example:

If the user searches for 90, the function should return 2.

If the user searches for 4, the function should return -1.

Sample Input/Output:

Input:

Enter a number to find: 90

Output:

The number 90 is at index 2.

Input:

Enter a number to find: 7

Output:

The number 7 is not in the array.

Q3.

Define a **structure named course** to store information related to a university course. The structure should include the course code, credit hours, GPA, attendance marks (out of 10), assignment marks (out of 20), five quiz marks stored in an array (each out of 20), midterm marks (out of 20), and final examination marks (out of 30).

In the **main()** function, first take an integer input **N** from the user and then declare an array of **N course structures**. For each course, take input for all required information except GPA.

After taking the inputs, write a user-defined function that determines the **best three quiz marks** from the three quizzes and converts them into a total of **20 marks**. Using this result, calculate the total marks obtained in each course by summing attendance, assignment, converted quiz marks, midterm, and final marks.

Next, write another user-defined function to determine the **GPA for each course** based on the **BRAC University grading policy** and store it in the corresponding structure.

Finally, calculate and print the **CGPA** using the formula:

$$CGPA = \frac{(GPA_1 \times Credit_1) + (GPA_2 \times Credit_2) + \dots}{Credit_1 + Credit_2 + \dots}$$

Print the course-wise details in the following format:

```
course_code - credit_hours - GPA
course_code - credit_hours - GPA
course_code - credit_hours - GPA
=====
Earned: X credits and CGPA: Y.YY
```

Note: If a course GPA is 0.00 (Fail), its credit hours should **not** be included in earned credits. However, failed course credits **must be included** in CGPA calculation.

Q4.

Trace the following **C program** and show the output using a proper tracing table:

```
#include <stdio.h>

int x = 3;
int total = 0;

void modify(int *p, int x){
    int temp = *p;
    int total = 5;
    *p = *p + x;
    x = x + 1;
    total = total + *p;
    printf("%d,%d,%d\n", x, *p, total);
    *p = temp;
}

int process(int x){
    int y = x;
    y = y + 2;
    return y;
}

void main(){
    int i;
    int x = 10;
    int y = 4;
    int *ptr = &y;

    for(i = 0; i < 5; i++){
        if(i == 2 || i == 4){
            total = total + x;
        }

        y = y + i;
        modify(ptr, x);

        x = process(x);

        if(i % 2 == 1){
            *ptr = *ptr + total;
        }
    }
}
```

```
    }

    printf("%d:%d:%d:%d\n",i, x, y, total);
}

printf("Final values:\n");
printf("%d:%d\n", x, y);
printf("total = %d\n", total);
}
```